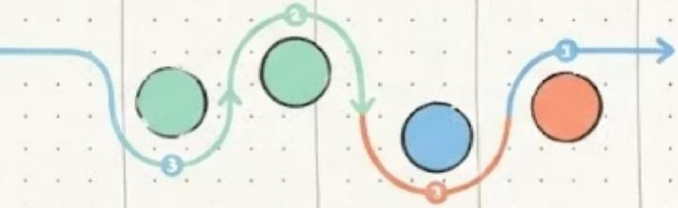


Gestão do Ciclo de Vida da Aplicação

Testabilidade

Professor Romário da Silva Borges



Testabilidade

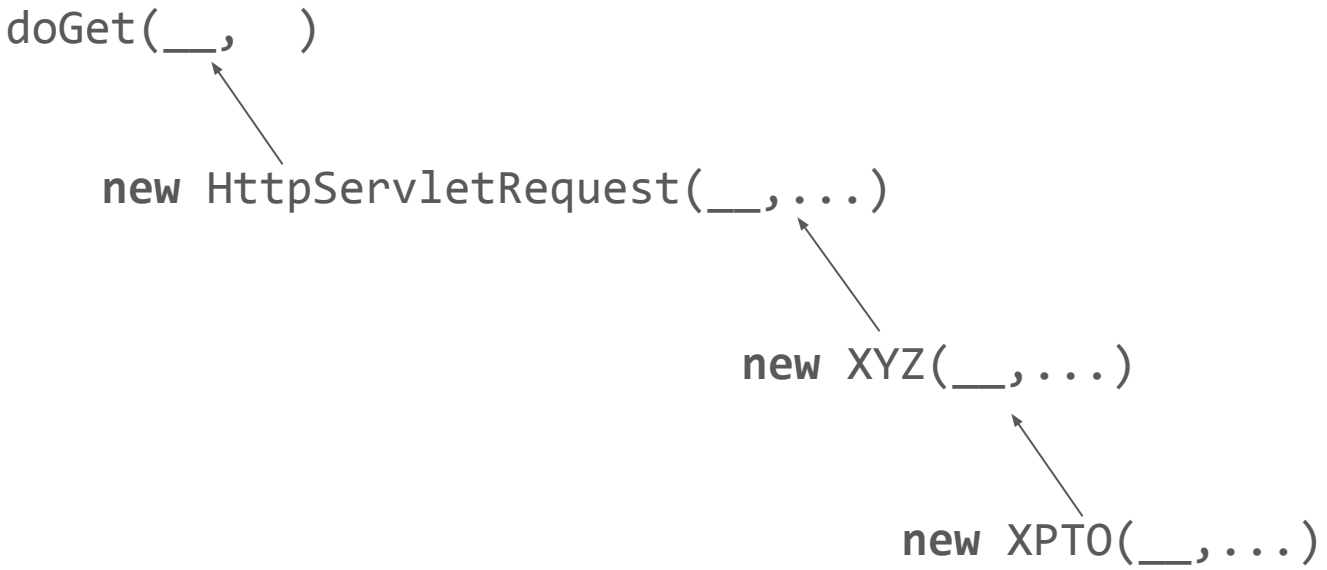
Exemplo: Servlet

```
public class IMCServlet extends HttpServlet {

  public void doGet(HttpServletRequest req,
                    HttpServletResponse res) {
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
    String peso = req.getParameter("peso");
    String altura = req.getParameter("altura");
    try {
      double p = Double.parseDouble(peso);
      double a = Double.parseDouble(altura);
      double imc = p / (a * a);
      out.println("Índice de Massa Corporal (IMC): "+imc);
    } catch (NumberFormatException e) {
      ...
    }
  }
}
```

Difícil testar **doGet**, pois ele tem dependências (parâmetros) para o pacote de Servlets de Java

Problema: chamar doGet(...) não é fácil…



Ou seja, testabilidade de doGet() é baixa

```
public class IMCServlet extends HttpServlet {

  public void doGet(HttpServletRequest req,
                    HttpServletResponse res) {
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
    String peso = req.getParameter("peso");
    String altura = req.getParameter("altura");
    try {
      double p = Double.parseDouble(peso);
      double a = Double.parseDouble(altura);
      double imc = p / (a * a);
      out.println("Índice de Massa Corporal (IMC): "+imc);
    } catch (NumberFormatException e) {
      ...
    }
  }
}
```



```
class IMCModel {

   public double calculaIMC(String p1, String a1){
     double p = Double.parseDouble(p1);
     double a = Double.parseDouble(a1);
     return p / (a * a);
  }
}
```

Solução: extração da regra de domínio para uma classe separada (e mais fácil de testar)

```
public class IMCServlet extends HttpServlet {
  IMCModel model = new IMCModel();

  public void doGet(HttpServletRequest req, HttpServletResponse res) {
    ...
    try {
      double imc = model.calculaIMC(peso, altura);
      out.println("Índice de Massa Corporal (IMC): " + imc);
    } …
  }
```

Mocks

Exemplo motivador

```
public interface BookService {
  String search(int isbn);
}


public class BookSearch {
  BookService rbs;

  public BookSearch(BookService rbs) {
    this.rbs = rbs;
  }

  public Book getBook(int isbn) {
    String json = rbs.search(isbn);
    JSONObject obj = new JSONObject(json);
    String titulo = (String) obj.get("titulo");
    return new Book(titulo);
  }
}
```

Método que queremos testar

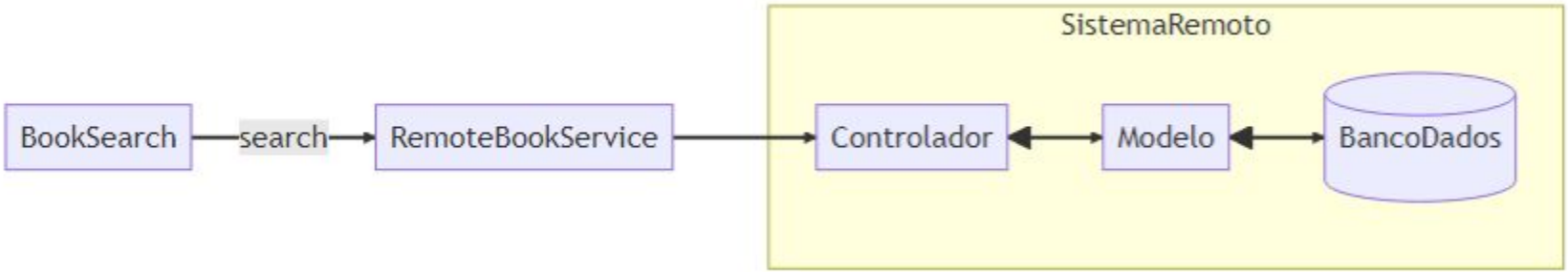
Acessa um serviço externo

Problema:
Testes de unidade devem ser rápidos!

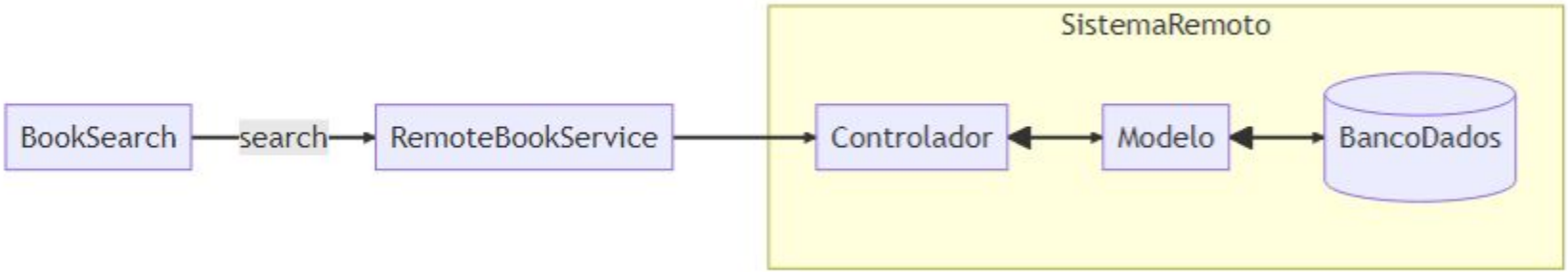
Solução: Mocks

- Objeto que emula um objeto real
- Porém, é bem mais simples que o objeto real

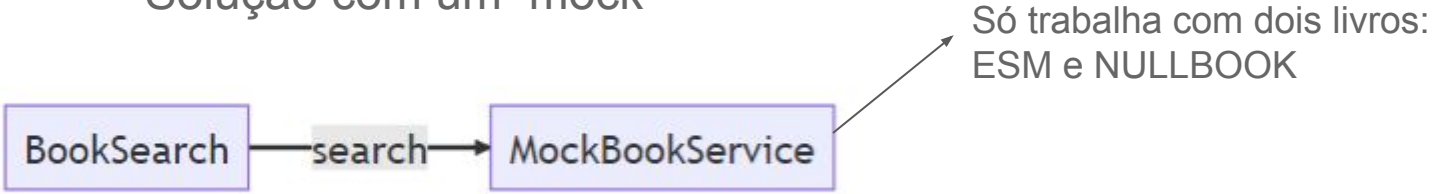
Solução sem mocks (teste lento, pois acessa um servidor remoto)



Solução sem mocks (teste lento, pois acessa um servidor remoto)



Solução com um mock



```
class BookConst {
  public static String ESM = "{ \"titulo\": \"Eng Soft Moderna\" }";
  public static String NULLBOOK = "{ \"titulo\": \"NULL\" }";
}

class MockBookService implements BookService {

   public String search(int isbn) {
      if (isbn == 1234)
         return BookConst.ESM;
      return BookConst.NULLBOOK;
   }

}
```



Simula pesquisa em um único ISBN

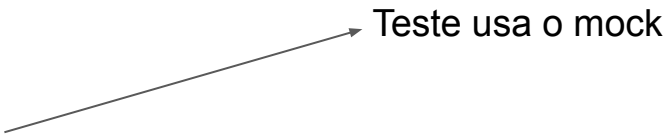
```
public class BookSearchTest {

  private BookService service;

  @Before
  public void init() {
    service = new MockBookService();
  }

  @Test
  public void testGetBook() {
    BookSearch bs = new BookSearch(service);
    String titulo = bs.getBook(1234).getTitulo();
    assertEquals("Eng Soft Moderna", titulo);
  }

}
```



```
public class IMCServlet extends HttpServlet {

  IMCModel model = new IMCModel();

  public void doGet(HttpServletRequest req,
                    HttpServletResponse res) {
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
    String peso = req.getParameter("peso");
    String altura = req.getParameter("altura");
    double imc = model.calculaIMC(peso,altura);
    out.println("IMC: " + imc);
  }

}
```

```
public class IMCServletTest {

  HttpServletRequest req;
  HttpServletResponse res;

  StringWriter sw;

  @Before
  public void init() {
    req = Mockito.mock(HttpServletRequest.class);
    res = Mockito.mock(HttpServletResponse.class);
    sw = new StringWriter();
    PrintWriter pw = new PrintWriter(sw);
    when(res.getWriter()).thenReturn(pw);
  }
  // ...continua a seguir
```

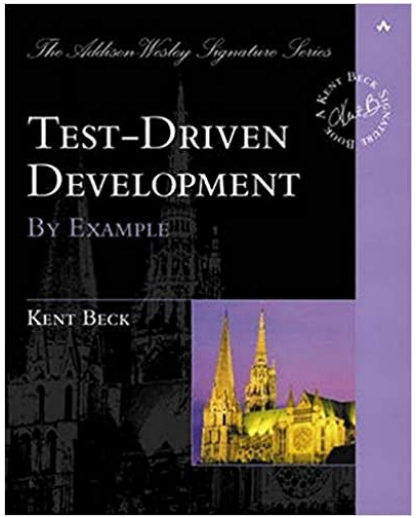
```
  // continuação de IMCServletTest
  @Test
  public void testDoGet() {
    when(req.getParameter("peso")).thenReturn("82");
    when(req.getParameter("altura")).thenReturn("1.80");
    new IMCServlet().doGet(req,res);
    assertEquals("IMC: 25.3\n", sw.toString());
  }

}
```

Desenvolvimento Dirigido por Testes (TDD)

TDD

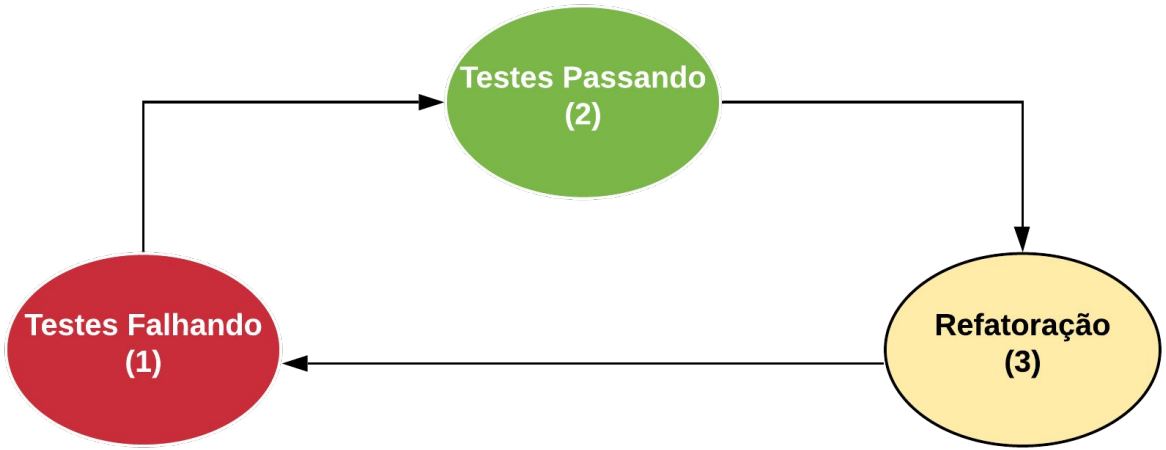
- Uma das práticas de programação propostas por XP
- Ideia simples: escrever o teste T antes da classe C



Benefícios

- Evita que os devs esqueçam de escrever os testes
- Incentiva a escrita de código com testabilidade
- Melhora o design do código, pois o dev passa a ser o primeiro cliente do seu código
- Cobertura pode chegar a 90%

Ciclo TDD



Exemplo de TDD: carrinho de compras

Começamos definindo que **ShoppingCart** terá um método add e um método **getTotal**. Além de decidir o nome de tais métodos, definimos os seus parâmetros e escrevemos o primeiro teste.

Vermelho




```
@Test
void testAddGetTotal() {
  Book b1 = new Book("book1", 10, "1");
  Book b2 = new Book("book2", 20, "2");
  ShoppingCart cart = new ShoppingCart();
  cart.add(b1);
  cart.add(b2);
  assertEquals(30.0, cart.getTotal());
}
```

Ainda vermelho, mas pelo menos compilando

```
public class Book {
  public String title;
  public double price;
  public String isbn;

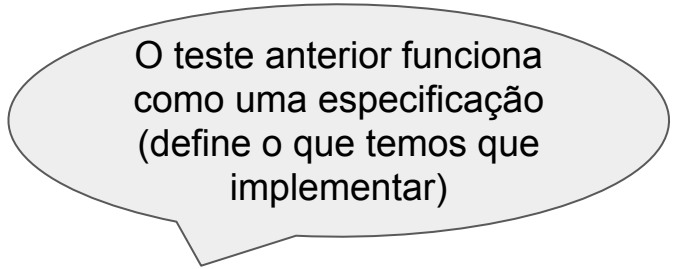
  public Book(String title, double price, String isbn) {
    this.title = title;
    this.price = price;
    this.isbn = isbn;
  }
}

public class ShoppingCart {
  public ShoppingCart() {}
  public void add(Book b) {}
  double getTotal() {
    return 0.0;
  }
}
```



Implementação provisória

Primeiro Verde




```
public class ShoppingCart {

  public ShoppingCart() {}

  public void add(Book b) {}

  double getTotal() {
    return 30.0;
  }
}
```

pequena vitória… **baby steps**

```
public class ShoppingCart {

  private ArrayList<Book> items;

  private double total;

  public ShoppingCart() {
    items = new ArrayList<Book>();
    total = 0.0;
  }

  public void add(Book b) {
    items.add(b);
    total += b.price;
  }

  double getTotal() {
    return total;
  }

}
```

Agora um verde mais real

Amarelo: Onde podemos refatorar e melhorar o código?

```
public class Book {
  public String title;
  public double price;
  public String isbn;

  public Book(String title, double price, String isbn) {
    this.title = title;
    this.price = price;
    this.isbn = isbn;
  }
}
```

Amarelo: podemos refatorar e melhorar o código?

```
public class Book {
  private String title;
  private double price;
  private String isbn;

  public Book(String title, double price, String isbn) {
    this.title = title;
    this.price = price;
    this.isbn = isbn;
  }
}
```



Encapsular esses campos

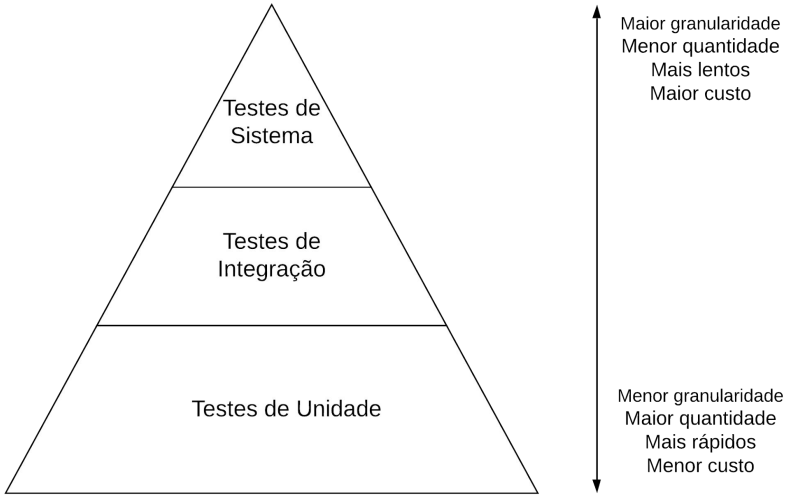
Próximo passo?

- Precisamos de mais alguma funcionalidade?
- Se sim: mais um ciclo TDD (vermelho-verde-amarelo)

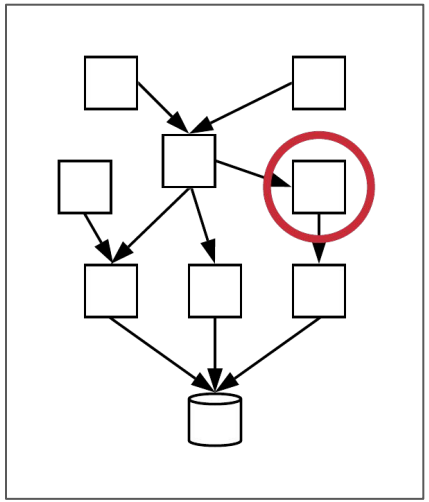
Teste de Integração

Testes de Integração

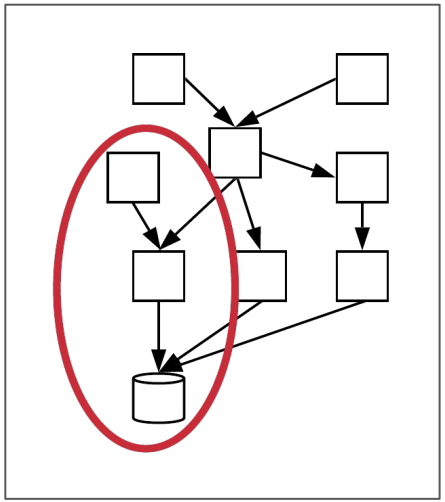
- Testam uma funcionalidade ou serviço do seu sistema
- Incluindo classes e serviços externos (BD, por exemplo)



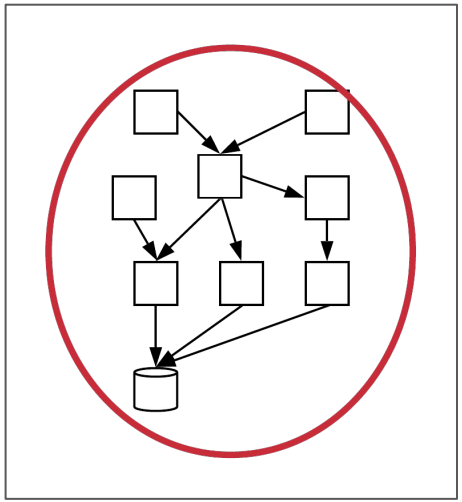
Relembrando ...



Unidade

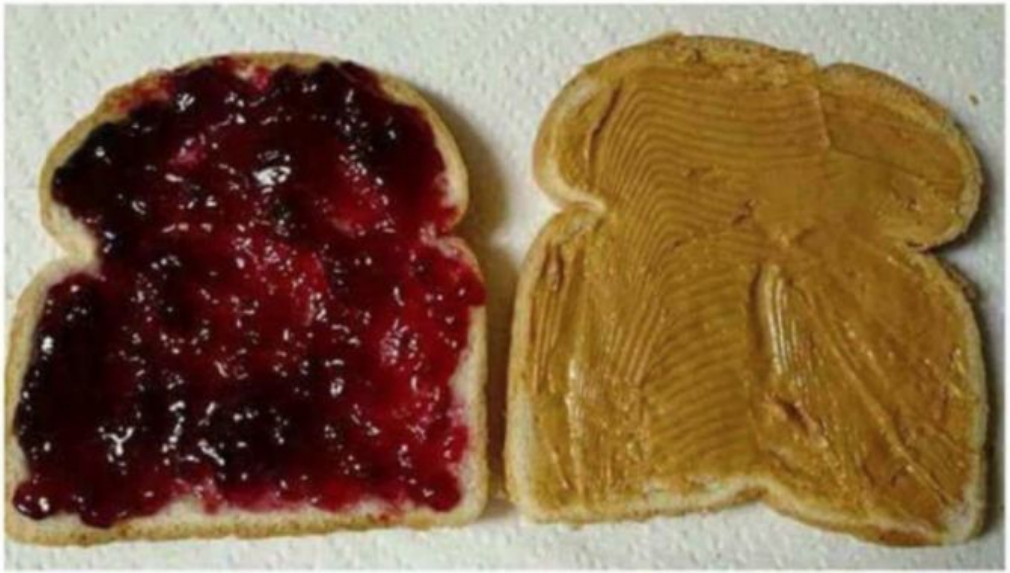


Integração



Sistema

Dois testes de unidade, zero testes de integração





Exemplo de Teste de Integração: ESM Forum

https://github.com/mtov/esmforum

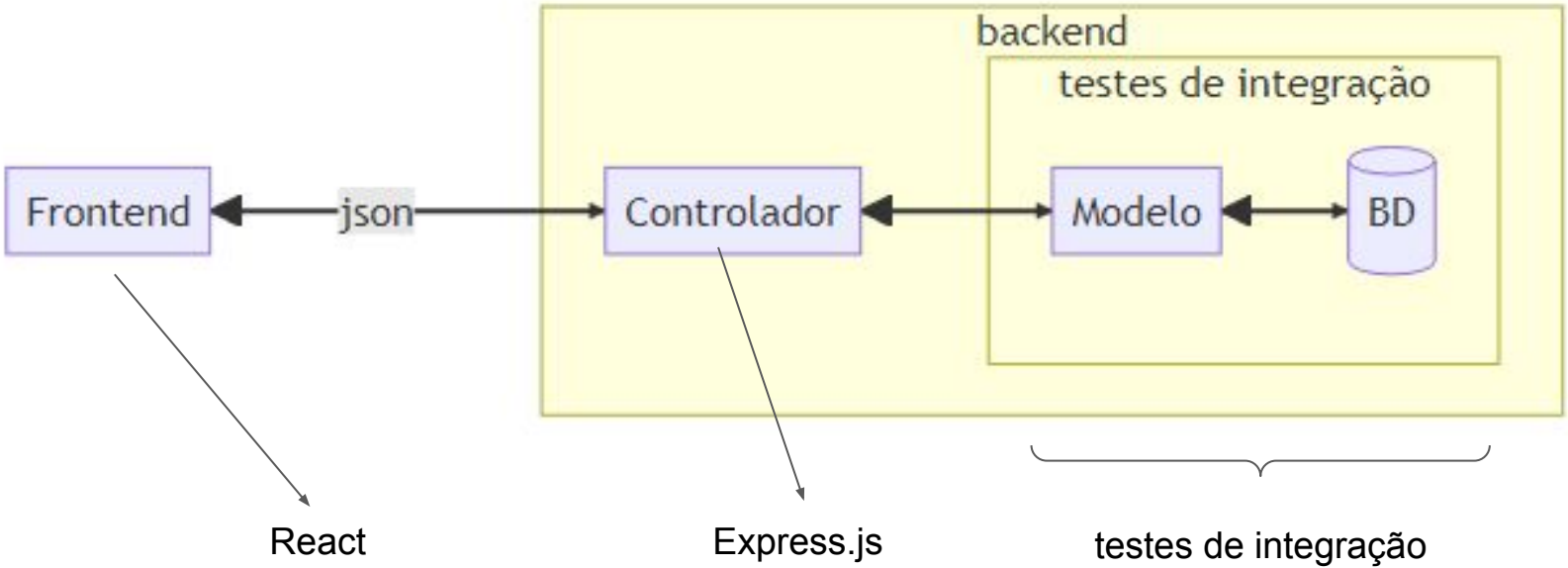
ESM Forum Perguntas Sobre

Perguntas Atuais

ID	Pergunta	Respostas
1	Como eu faço para remover um determinado elemento de um array em JavaScript?	2
2	Como verificar se uma string contém uma substring em JavaScript?	0
3	Como remover uma propriedade de um objeto JavaScript?	0
4	Em JavaScript, qual é a diferença entre "let" e "var"?	0
5	Qual operador de igualdade (isto é: "==" ou "===") deve ser usado em JavaScript?	0

Faça sua pergunta

Enviar


backend
testes de integração
Frontend
json
Controlador
Modelo
BD
React
Express.js
testes de integração

Jest

```
const bd = require('../bd/bd_utils.js');
const modelo = require('../modelo.js');

beforeEach(() => {
  bd.reconfig('./bd/esmforum-teste.db');

  // limpa todas as tabelas
  bd.exec('delete from perguntas', []);
  bd.exec('delete from respostas', []);
});

test('Testando banco de dados vazio', () => {
  expect(modelo.listar_perguntas().length).toBe(0);
});
```

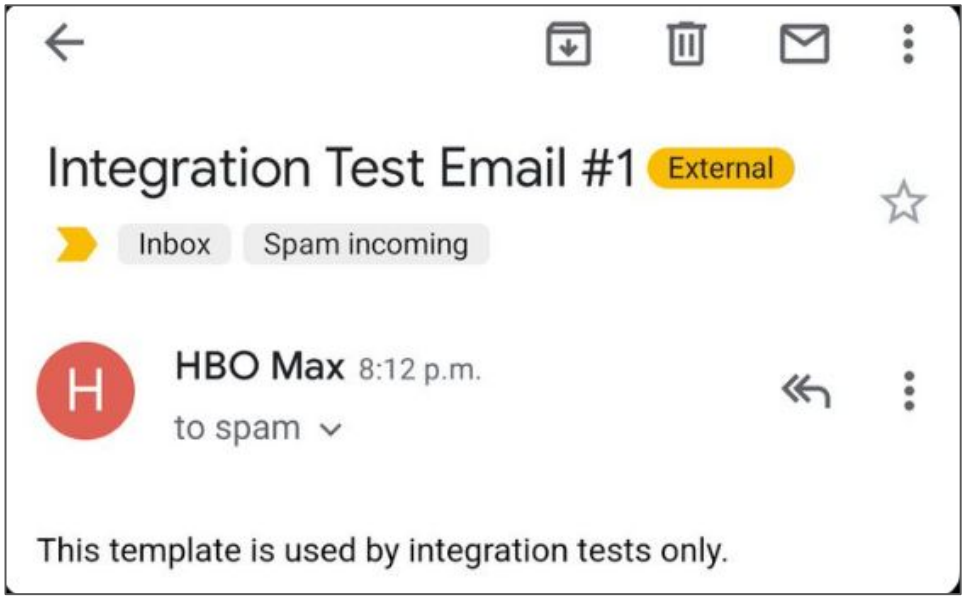
Jest

```
test('Testando cadastro de três perguntas', () => {
  modelo.cadastrar_pergunta('1 + 1 = ?');
  modelo.cadastrar_pergunta('2 + 2 = ?');
  modelo.cadastrar_pergunta('3 + 3 = ?');

  const perguntas = modelo.listar_perguntas();

  expect(perguntas.length).toBe(3);
  expect(perguntas[0].texto).toBe('1 + 1 = ?');
  expect(perguntas[1].texto).toBe('2 + 2 = ?');
  expect(perguntas[2].num_respostas).toBe(0);
});
```

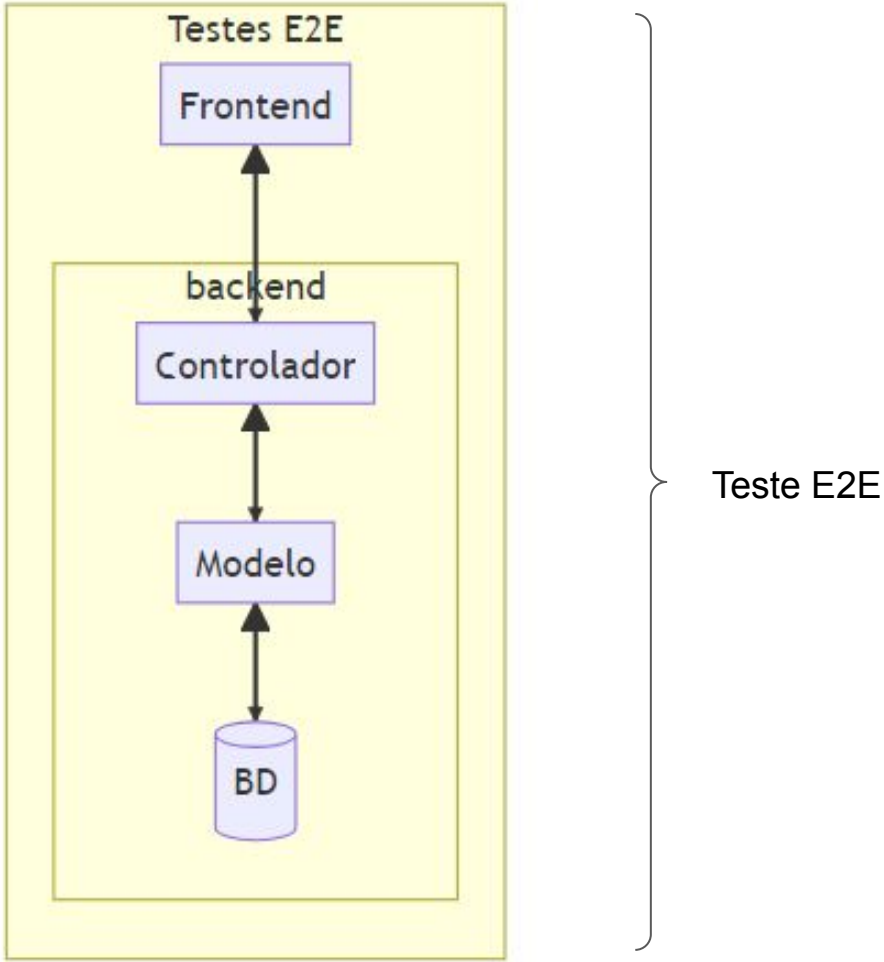
1. Em junho de 2021, o seguinte e-mail foi enviado por engano para milhares de assinantes da HBO. O que deve ter acontecido para que este mail chegasse aos usuários finais da HBO?



Testes de Sistema

Testes de Sistema

- Testam o sistema inteiro; via sua interface externa
- Também chamados de:
  - Testes ponta-a-ponta (End-to-End ou E2E)
  - Testes de frontend


Testes E2E
Frontend
backend
Controlador
Modelo
BD
Teste E2E



Exemplo de Teste E2E



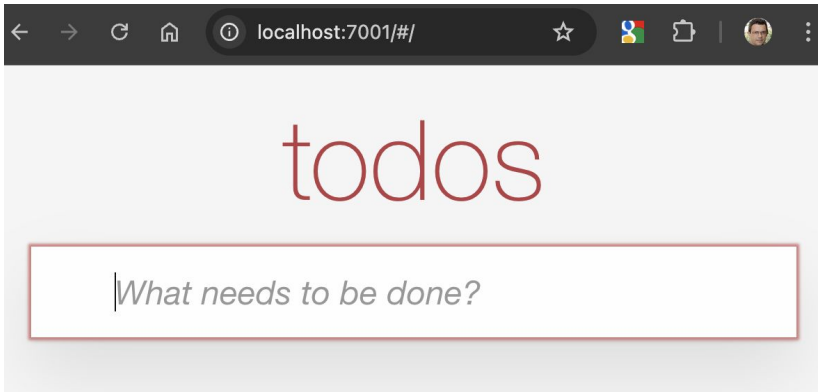
https://todomvc.com


todo-input (input)
todos-list (container)
toggle-todo-checkbox
todos
What needs to be done?
Trabalho Prático Testes E2E
Prova de Sistemas Operacionais
Trabalho Prático de Compiladores
Prova de Cálculo III
3 items left
All
Active
Completed
Clear completed
remove-todo-btn
(visible on hover)
filter-all-link
filter-active-link
filter-completed-link

HTML & data-cy Selectors

```
<input data-cy="todo-input" ... placeholder="What needs to be done?"/>
...
<input data-cy="toggle-todo-checkbox" type="checkbox" {{checked}}>
...
<button data-cy="remove-todo-btn" class="destroy"></button>
...
<ul class="filters">
  <li> <a data-cy="filter-all-link" href="#/" ...">All</a> </li>
  <li> <a data-cy="filter-active-link" href="#/active">Active</a> </li>
  <li> <a data-cy="filter-completed-link"
          href="#/completed">Completed</a> </li>
</ul>
```

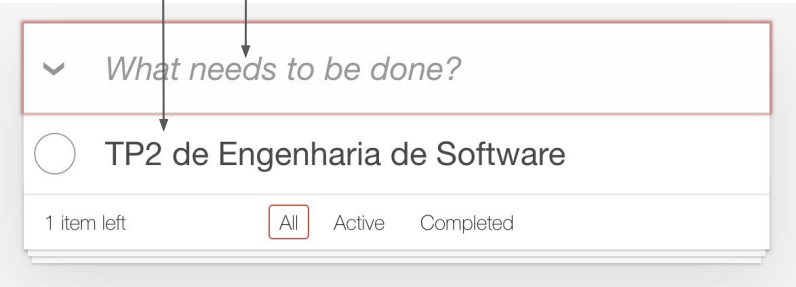
```
it('Verifica se app está abrindo', () => {
    cy.visit('')
  })
```



```
it('Insere uma tarefa', () => {
    cy.visit('');

    cy.get('[data-cy=todo-input]')
      .type('TP2 de Engenharia de Software{enter}');

    cy.get('[data-cy=todos-list]')
      .children()
      .should('have.length', 1)
      .first()
      .should('have.text', 'TP2 de Engenharia de Software');
  });
```



```
it('Insere e deleta uma tarefa', () => {
    cy.visit('');

    cy.get('[data-cy=todo-input]')
      .type('TP2 de Engenharia de Software{enter}');

    cy.get('[data-cy=todos-list]')
      .children()
      .should('have.length', 1);
```

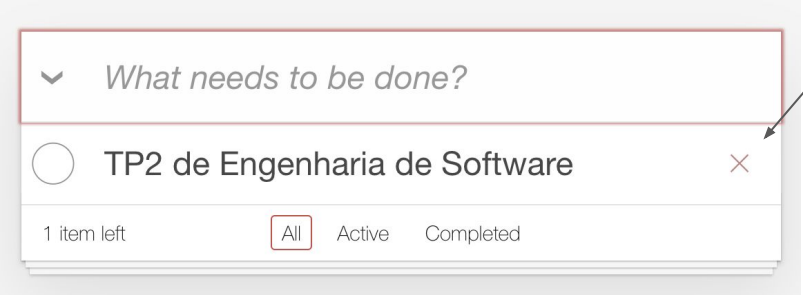
Até aqui sem novidades para o teste anterior

Continua…

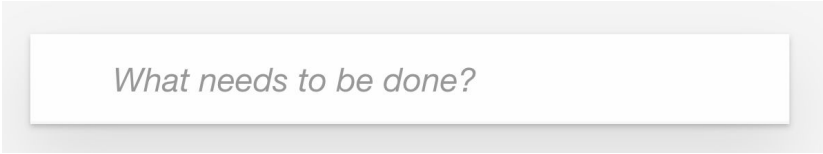
Continuação…

```
  cy.get('[data-cy=todos-list] > li [data-cy=remove-todo-btn]')
    .invoke('show')
    .click();

  cy.get('[data-cy=todos-list]')
    .children()
    .should('have.length', 0);
});
```



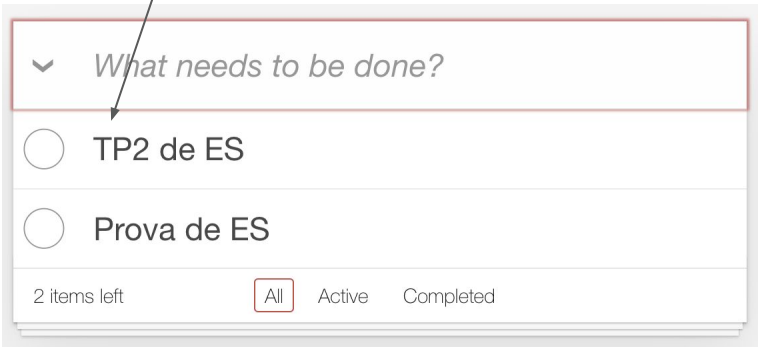


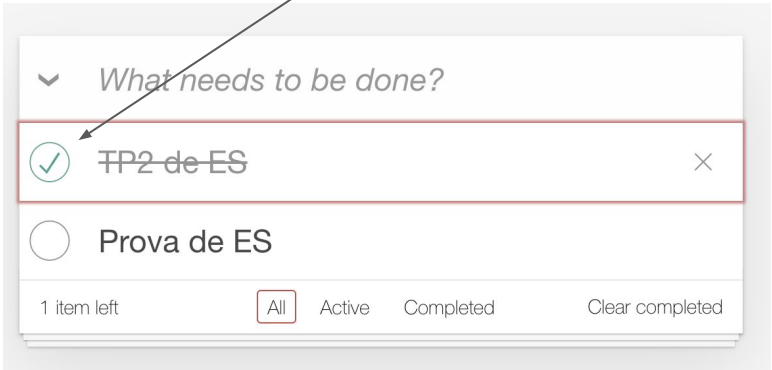


```
it('Filtra tarefas completas e ativas', () => {
    cy.visit('');

    cy.get('[data-cy=todo-input]')
      .type('TP2 de ES{enter}')
      .type('Prova de ES{enter}');

    cy.get('[data-cy=todos-list] > li [data-cy=toggle-todo-checkbox]')
      .first()
      .click();
```





Continua…

Continuação…

```
cy.get('[data-cy=filter-active-link')
  .click();

cy.get('[data-cy=todos-list]')
  .children()
  .should('have.length', 1)
  .first()
  .should('have.text', 'Prova de ES');
```

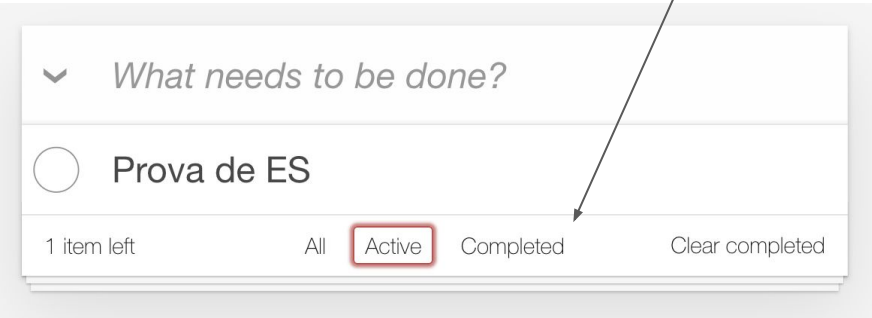


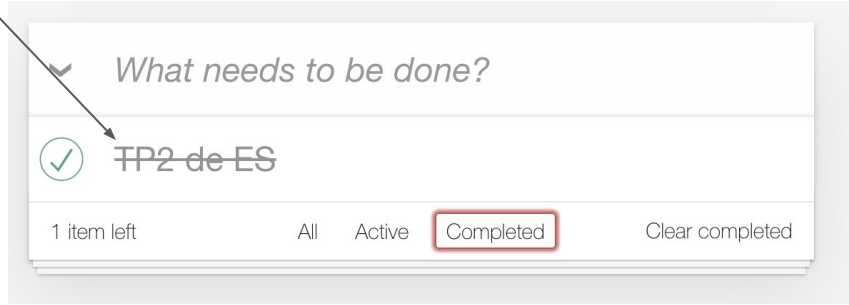
Continua…

Continuação…

```
cy.get('[data-cy=filter-completed-link')
  .click();

cy.get('[data-cy=todos-list]')
  .children()
  .should('have.length', 1)
  .first()
  .should('have.text', 'TP2 de ES');
```



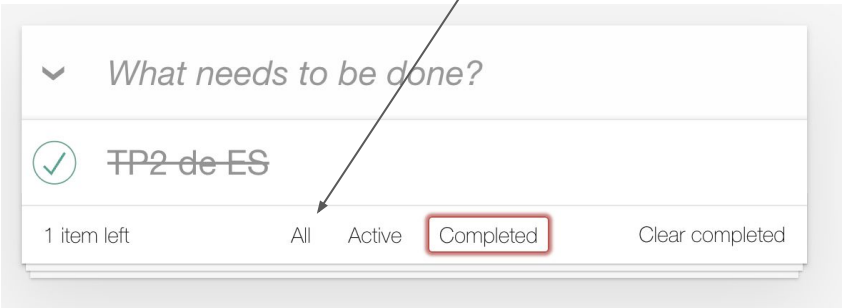


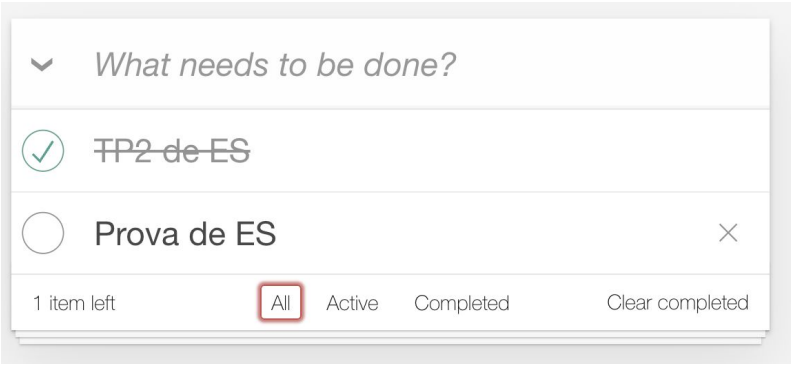
Continua…

Continuação…

```
    cy.get('[data-cy=filter-all-link')
      .click();

    cy.get('[data-cy=todos-list]')
      .children()
      .should('have.length', 2);
  });
});
```





Roteiro prático de uso do Cypress:

https://github.com/mtov/roteiro-cypress

Fuzz Testing

- Técnica na qual um programa é executado com entradas geradas automaticamente, inesperadas ou aleatórias
- Objetivo: encontrar falhas ou vulnerabilidades de segurança



Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: https://engsoftmoderna.info.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

